

Organização e Arquitetura de Computadores

Introdução a Assembly

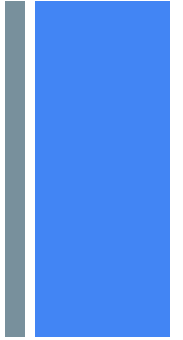
Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

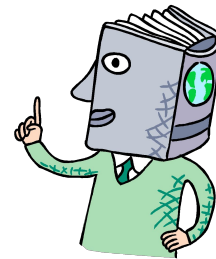
Campinas, 1s/2026



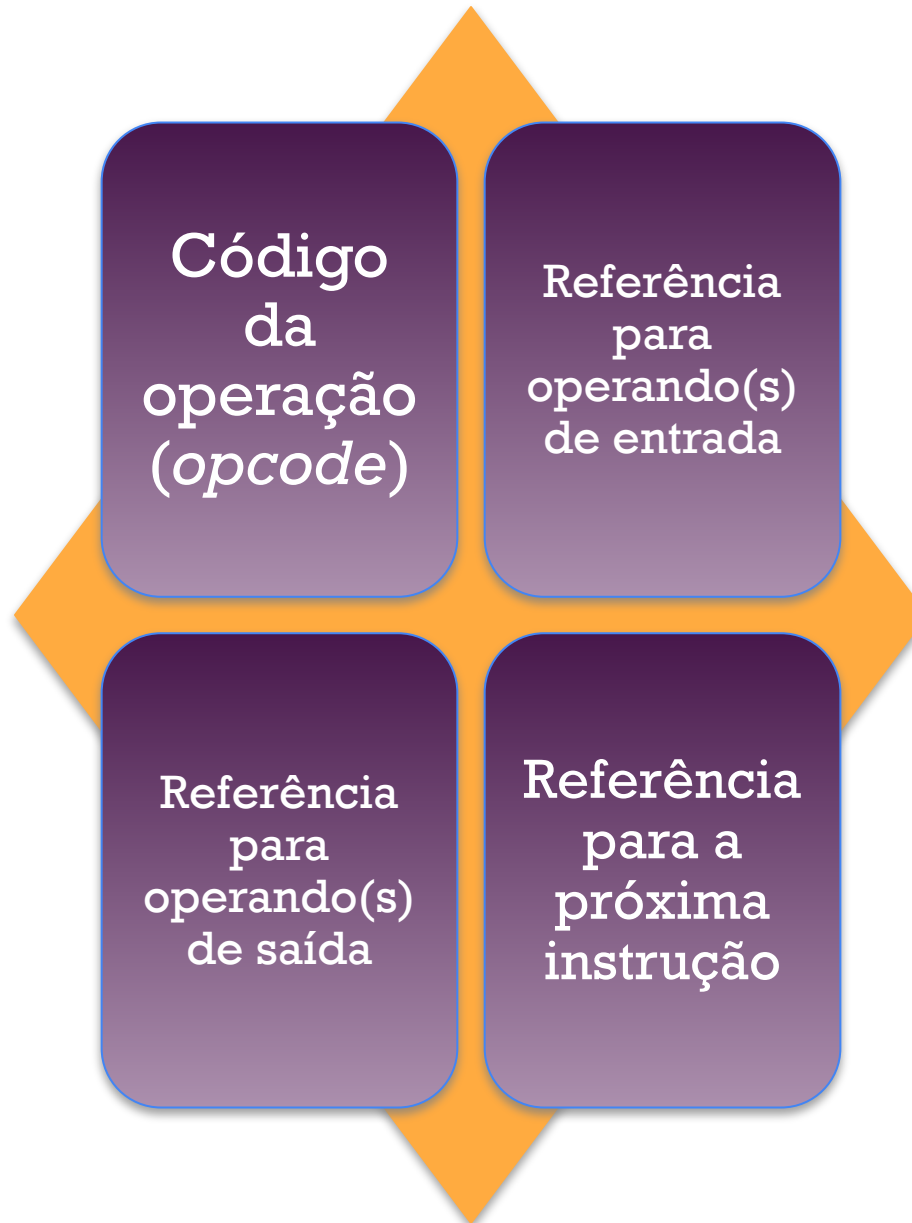
Características de instruções



- A operação do processador é determinada pelas instruções que ele executa, chamadas de *instruções de máquina* ou *instruções de computador* (ou, aqui, somente **instruções**)
- A coleção de instruções que um processador pode executar é o *conjunto de instruções* (**ISA**: *Instruction Set Architecture*)
- Cada instrução carrega uma quantidade de informação requerida pelo processador para executar alguma operação



Elementos de uma instrução



Operandos podem estar em:



1) Memória

- Instrução contém o endereço de memória para o operando

2) Dispositivo de E/S

- Instrução contém uma codificação identificando o módulo de E/S utilizado
- É comum que E/S seja mapeada como uma região de memória

3) Registrador interno

- O processador possui registradores que podem ser usados como operandos
- Se houver mais do que um registrador, a instrução contém um código que identifica qual é usado como operando

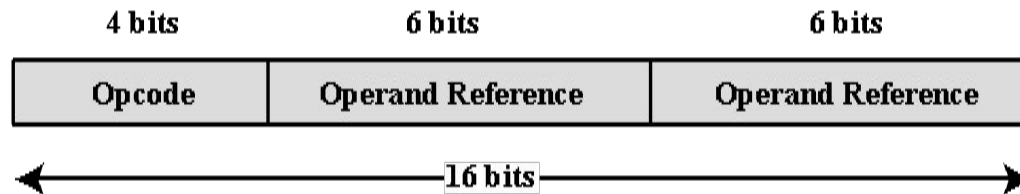
4) Imediato

- O valor do operando está codificado diretamente na instrução



Representação de uma instrução

- No computador, uma instrução é representada por uma sequência de bits
- A instrução é dividida em campos, correspondendo aos elementos que a constituem





Representação de uma instrução



- Opcodes são representados por abreviações chamadas de *mnemônicos*
- Exemplos
 - ADD *Add* - Somar
 - SUB *Subtract* - Subtrair
 - MUL *Multiply* - Multiplicar
 - DIV *Divide* - Dividir
 - LOAD *Load data from memory* – Carregar da memória
 - STOR *Store data to memory* – Salvar na memória
- Operandos também são representados por símbolos
 - ADD R, Y ou add \$t0, \$t1, \$t2
- Cada símbolo tem uma representação binária fixa

Instruction Types



Instruction Set Design



Muito complexo pois define muitos aspectos do sistema



Define muitas das funções do processador



É a maneira do programador controlar o sistema



Fundamental design issues:

Repertório de operações

- Quantas, quais e quão complexas?

Tipos de dados
• Com o que operar?

Formato de instruções

- Quantos campos e de qual tamanho?

Registradores

- Quantos registradores e para que servem?

Endereçamento

- Como especificar localizações de memória?

RISC X CISC

RISC (Reduced Instruction Set Computer)

- 1. Instruções simples**
- 2. Construído para ser simples e rápido**
- 3. Referências a memória só com LOAD/STORE**
- 4. Instruções de formato Fixo**
- 5. Poucas instruções com poucos modos de endereçamento**
- 6. Compilador complexo**
- 7. Vários registradores**

CISC (Complex Instruction Set Computer)

- 1. Instruções complexas**
- 2. Construído para se parecer com linguagens de alto nível**
- 3. Qualquer instrução pode referenciar à memória**
- 4. Instruções com formato variável**
- 5. Muitas instruções com muitos modos de endereçamento**
- 6. A complexidade está no programa**
- 7. Poucos registradores**

Instruções MIPS

- Utilizado aqui como exemplo
- Stanford MIPS comercializado pela MIPS Technologies (www.mips.com)
- Utilizado principalmente em embarcados
- Bem representativo de ISAs modernos

`addw a0,a0,a1`

`addu $2,$4,$5`

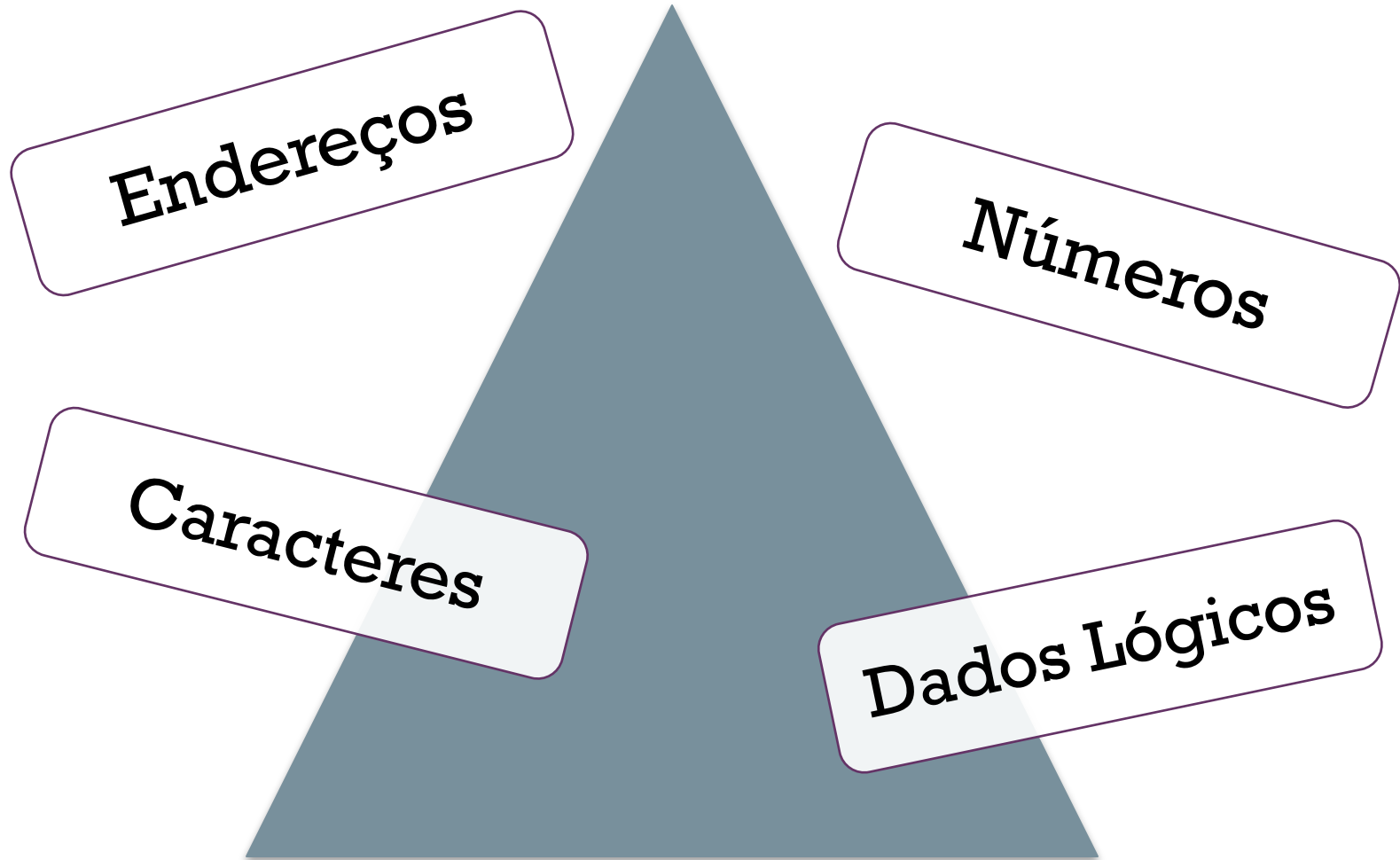
`add %o0,%o1,%o0`

`add 3,3,4`

`add w0,w1,w0`

`lea eax,[rdi+rsi]`

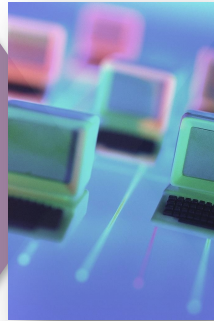
Types of Operands



Data Transfer



O tipo mais
fundamental de
instrução

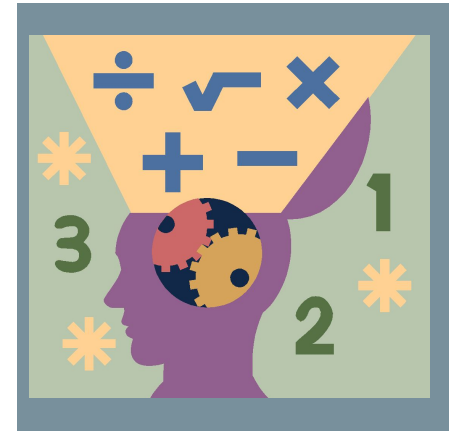


Deve especificar

- Localização dos operandos
- Tamanho dos dados
- Modo de endereçamento de cada operando



- A maioria das máquinas oferecem operações aritméticas básicas ($+$, $-$, $*$, $/$)
- Operações básicas trabalham com números inteiros com sinal
- Podem também utilizar ponto flutuante, BCD, etc.
- Outras operações incluem:
 - Valor absoluto
 - Negação
 - Incremento
 - Decremento



Aritmética

Operações Aritméticas

- Soma e subtração, 3 operandos

- Duas entradas e uma saída

add a, b, c # $a \leftarrow b + c$

sub a, b, c # $a \leftarrow b - c$

- Todas as operações aritméticas seguem esse formato

Exemplo aritmético

- Código em C:

```
f = (g + h) - (i + j);
```

Exemplo aritmético

- Código em C:

```
f = (g + h) - (i + j);
```

- Código MIPS compilado:

```
add x, g, h    # temp x = g + h  
add y, i, j    # temp y = i + j  
sub f, x, y    # f = x - y
```

Operandos em registradores

- Instruções aritméticas usam operandos em registradores
- MIPS tem 32 registradores de 32 bits cada
 - Utilizados para dados frequentemente acessados
 - Numerados de 0 a 31
 - Dados de 32 bits são uma “palavra”
 - R0 é sempre zero (\$zero)
- Nomes no *assembler*
 - \$t0, \$t1, ..., \$t9 para valores temporários
 - \$s0, \$s1, ..., \$s7 para variáveis salvas

Exemplo com registradores

- Código em C:

`f = (g + h) - (i + j);`

f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

Exemplo com registradores

- Código em C:

$f = (g + h) - (i + j);$

f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

- Código MIPS compilado:

add \$t0, \$s1, \$s2

add \$t1, \$s3, \$s4

sub \$s0, \$t0, \$t1

Operandos em memória

- Memória principal utilizada para dados estruturados
 - Vetores, estruturas, alocação dinâmica
- Para fazer operações aritméticas
 - Carregar (*load*) valores da memória em registradores
 - Guardar (*store*) os resultados de volta na memória
- Memória endereçada por bytes
 - Cada endereço aponta para um dado de 8 bits
- Palavras alinhadas na memória
 - Endereço deve ser um múltiplo de 4

```
lw    $s1, 20($s2)    # $s1 ← MEMÓRIA($s2 + 20)
```

```
sw    $s1, 20($s2)    # MEMÓRIA($s2 + 20) ← $s1
```

deslocamento

base

Exemplo com memória

- Código em C:
 $g = h + A[8];$

g	\$s1
h	\$s2
&A	\$s3

Exemplo com memória

- Código em C:

`g = h + A[8];`

g	\$s1
h	\$s2
&A	\$s3

- Código MIPS compilado:

- Índice 8 é um deslocamento de 32 bytes

- 4 bytes por palavra

`lw $t0, 32($s3) # load`

`word`

`add $s1, $s2, $t0`

Exemplo 2 com memória

- Código em C:

`A[12] = h + A[8];`

h	\$s2
&A	\$s3

Exemplo 2 com memória

- Código em C:

$A[12] = h + A[8];$

h	\$s2
&A	\$s3

- Código MIPS compilado:

```
lw    $t0, 32($s3)    # load
word
add   $t0, $s2, $t0
sw    $t0, 48($s3)    # store
word
```

Registradores vs Memória

- Registradores são mais rápidos de acessar
- Operar em memória requer *loads* e *stores*
 - Mais instruções para serem executadas
- O compilador deve usar registradores o máximo possível
 - Só partir para memória para variáveis menos utilizadas
 - Otimizar o uso de registradores é importante!

Operandos em imediatos

- Constantes embutidas na instrução
`addi $s3, $s3, 4`
- Sem subtração com imediato
 - Só usar uma constante negativa
`addi $s2, $s1, -1`
- Fazer o uso comum ser rápido
 - Pequenas constantes são comuns
 - Operandos em imediatos evitam um *load*

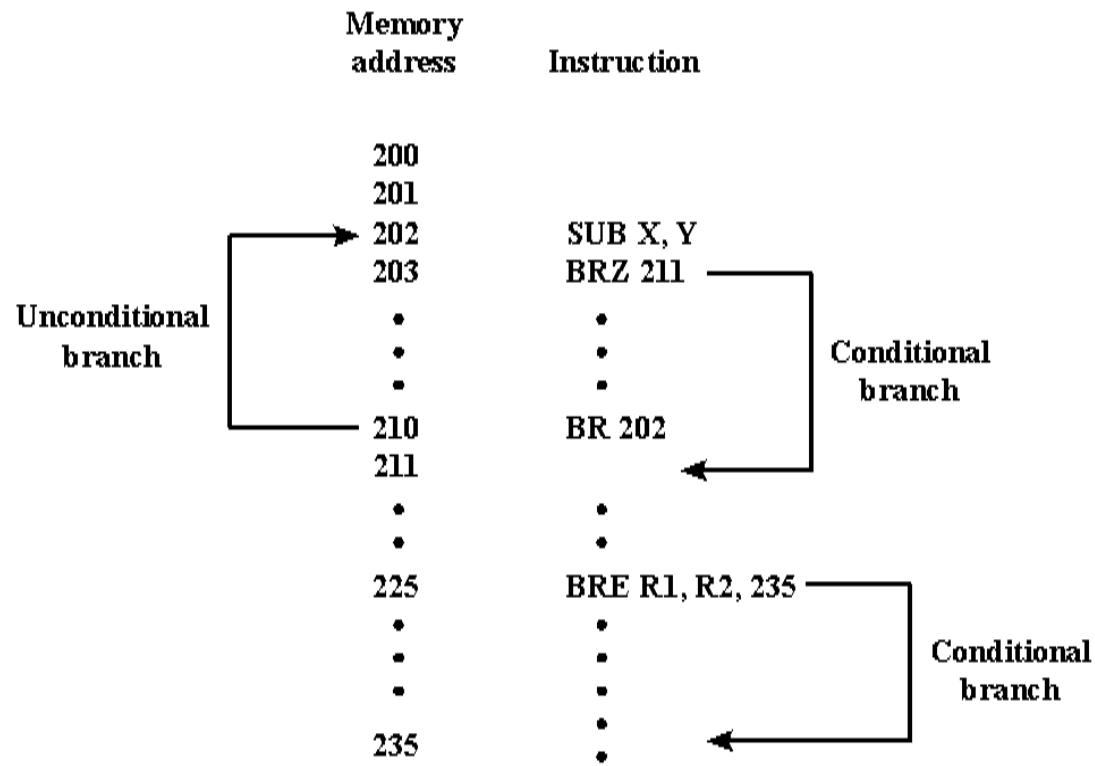
Operações lógicas

- Instruções para manipulação de bits

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR			or, ori
Bitwise NOT	~	~	nor

+ Transferência de controle

- Motivos para utilizar transferência de controle
 - É essencial poder executar cada instrução mais do que uma vez (**WHILE**)
 - Qualquer programa envolve alguma tomada de decisão (**IF**)
 - Ajuda se houver como dividir a tarefa em pedaços menores (**funções**)
- As operações mais comuns de transferência de controle são:
 - Desvio (*branch*)
 - Salto (*skip*) – pular a próxima instrução
 - Chamada de procedimento



Operações condicionais

- Desviar (*branch*) para um *label* se a condição for verdadeira
 - Se não, continuar sequencialmente
- `beq rs, rt, L1`
 - if (rs == rt) desviar para L1;
- `bne rs, rt, L1`
 - if (rs != rt) desviar para L1;
- `j L1`
 - sempre desviar para L1

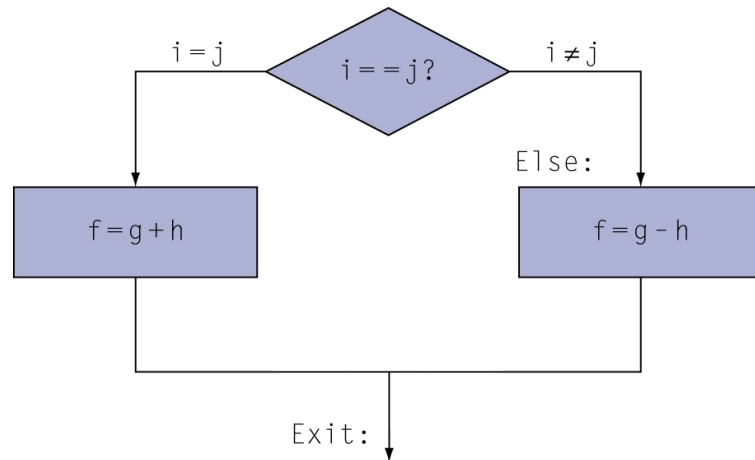
O *assembler* se encarrega de calcular os endereços dos *labels*

Compilando um IF

- Código em C:

```
if (i==j) f = g+h;  
else f = g-h;
```

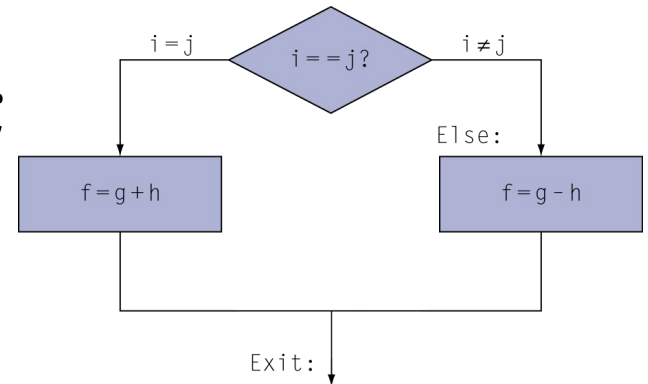
f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4



Compilando um IF

- Código em C:

```
if (i==j) f = g+h;  
else f = g-h;
```



- Código MIPS compilado:

f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

```
        bne $s3, $s4, Else  
        add $s0, $s1, $s2  
        j   Exit  
Else:   sub $s0, $s1, $s2  
Exit:   ...
```

Compilando um WHILE

- Código em C:

```
while (save[i] == k) i +=  
1;
```

i	\$s3
k	\$s5
&save	\$s6

Compilando um WHILE

- Código em C:

```
while (save[i] == k) i +=  
1;
```

- Código MIPS compilado:

```
Loop:  sll    $t1, $s3, 2  
       add    $t1, $t1, $s6  
       lw     $t0, 0($t1)  
       bne    $t0, $s5, Exit  
       addi   $s3, $s3, 1  
       j      Loop  
Exit:  ...
```

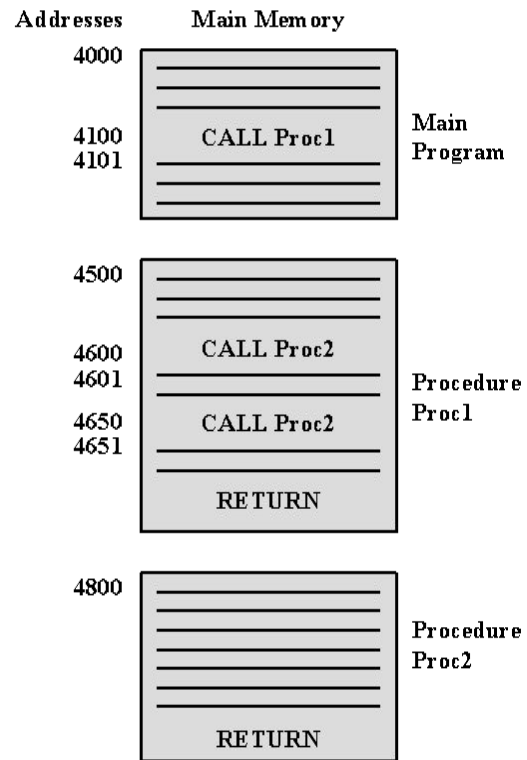
i	\$s3
k	\$s5
&save	\$s6

Mais operadores condicionais

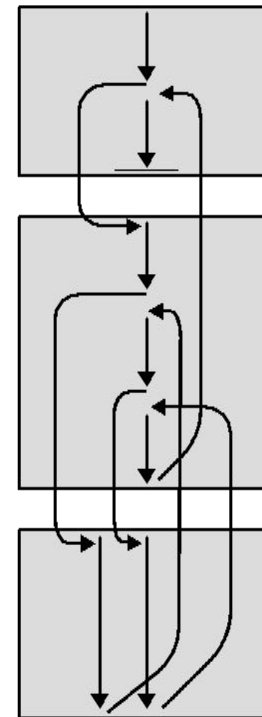
- Definir o resultado como 1 se a condição for verdadeira
 - Se não, definir como 0
- `slt rd, rs, rt`
 - if (rs < rt) rd = 1; else rd = 0;
- `slti rt, rs, const`
 - if (rs < const) rt = 1; else rt = 0;
- Uso juntamente com `beq`, `bne`
 - `slt $t0, $s1, $s2 # if ($s1 < $s2)`
 - `bne $t0, $zero, L # branch to L`

+ Chamadas de procedimento

- Programas menores incorporados em programas maiores
 - O procedimento pode ser “chamado” a qualquer momento
 - Processador desvia, executa o procedimento, e volta
- Dois principais motivos:
 - Economia
 - A mesma porção de código utilizada várias vezes
 - Modularidade
- Duas instruções básicas:
 - Uma “chamada” que desvia para o procedimento
 - Um “retorno” que volta para onde o procedimento foi chamado



(a) Calls and returns



(b) Execution sequence

Chamadas de procedimento

- Passos:
 1. Guardar os parâmetros em registradores
 2. Desviar para o procedimento
 3. Salvar registradores utilizados
 4. Executar as operações do procedimento
 5. Colocar resultado em registrador
 6. Restaurar os registradores originais
 7. Retornar

Convenção de registradores

- \$a0 – \$a3: argumentos
- \$v0, \$v1: resultados
- \$t0 – \$t9: temporários
 - Podem ser sobrescritos
- \$s0 – \$s7: salvos
 - Devem ser guardados e restaurados
- \$sp: ponteiro de pilha (*stack pointer*)
- \$ra: endereço de retorno

Instruções de procedimentos

- Chamada de procedimento:
jump and link
jal ProcedureLabel
 - Endereço da próxima instrução salvo em \$ra
 - Desvia para o endereço do *label*
- Retorno do procedimento: jump register
jr \$ra
 - Desvia para o endereço em \$ra

Exemplo de procedimento “folha”

- Código em C:

```
int leaf (int g, int h,  
int i,int j)  
{  
    int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

Exemplo de procedimento “folha”

- Código MIPS compilado:

leaf:

```
add    $t0, $a0, $a1
add    $t1, $a2, $a3
sub    $v0, $t0, $t1
jr     $ra
```

- Para chamar:

```
jal    leaf
```

Procedimentos “não-folha”

- Procedimentos que chamam outros procedimentos
- Devem salvar na pilha
 - Seu endereço de retorno (\$ra)
 - Registradores salvos que for usar
 - Quaisquer temporários e argumentos que precisar depois
 - Restaurar da pilha depois

Exemplo de procedimento “não-folha”

- Código em C:

```
void calcula (int *IN, int *OUT, int n) {  
    for (i=0; i<n; i++) {  
        OUT[i] = leaf (  
            IN[4*i+0], IN[4*i+1],  
            IN[4*i+2], IN[4*i+3]);  
    }  
}
```

Exemplo de procedimento “não-folha”

■ Código MIPS compilado:

calcula:

```
addi $sp, $sp, -20
sw    $s0, 0($sp)
sw    $s1, 4($sp)

sw    $s2, 8($sp)

sw    $s3, 12($sp)
sw    $ra, 16($sp)

add    $s0, $a0,
$zero
add    $s1, $a1,
$zero
add    $s2, $a2,
$zero
add    $s3, $zero,
$zero
sll    $s2, $s2, 2
```

Loop:

```
slt    $t0, $s3, $s2
beq    $t0, $zero,
Exit
sll    $t1, $s3, 2
add    $t1, $s0, $t1
lw     $a0, 0($t1)

lw     $a1, 4($t1)

lw     $a2, 8($t1)

lw     $a3, 12($t1)

jal    leaf
add    $t1, $s1, $s3

sw     $v0, 0($t1)

addi   $s3, $s3, 4
j      Loop
```

Exit:

```
lw     $s0, 0($sp)
lw     $s1, 4($sp)
lw     $s2, 8($sp)
lw     $s3, 12($sp)
lw     $ra, 16($sp)
addi   $sp, $sp, 20
jr     $ra
```